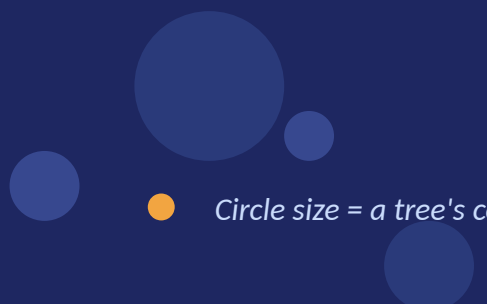




ENSEMBLE LEARNING

XGBoost

Extreme Gradient Boosting — regularized, parallel tree boosting built for speed and accuracy



● Circle size = a tree's contribution to the ensemble — the visual motif used throughout this deck

What Is Gradient Boosting?

Each new tree is trained to predict the residual errors of the ensemble so far — fitting the gradient of the loss, not the raw labels.



XGBoost (Chen & Guestrin, 2016) adds regularization, a second-order loss approximation, and system-level speed optimizations on top of this idea.

Core Idea

1

Additive trees

The model is a sum of many small trees, each one added to correct the current errors.

2

Second-order optimization

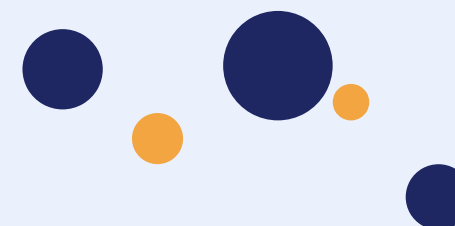
Uses both gradient and hessian of the loss for a more accurate step at each round.

3

Built-in regularization

Penalizes tree complexity directly in the objective — fewer leaves, smaller leaf weights.

Small trees,
added step by step



How the Algorithm Runs

1

Start with a base prediction

Usually the mean of the target (regression) or log-odds (classification).

2

Compute gradient & hessian

For every sample, the 1st and 2nd derivative of the loss w.r.t. the prediction.

3

Grow a tree greedily

Split leaves using the gain formula, pruned by regularization terms γ and λ .

4

Assign optimal leaf weights

Each leaf gets the weight that minimizes the regularized objective.

5

Shrink and add the tree

Scale the new tree's output by the learning rate η , then add it to the model.

6

Repeat & combine

Loop for $n_{\text{estimators}}$ rounds; the final model sums every tree's output.

The Key Formulas

The regularized objective XGBoost optimizes at every boosting round

1. Regularized objective

$$\text{Obj} = \sum l(y_i, \hat{y}_i) + \sum \Omega(f_k)$$

Training loss plus a penalty Ω on every tree's complexity.

2. Tree complexity penalty

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum w_j^2$$

Penalizes the number of leaves T and the size of leaf weights w_j .

3. Optimal leaf weight

$$w_j^* = -G_j / (H_j + \lambda)$$

G_j , H_j are summed gradients/hessians of samples in leaf j .

4. Split gain

$$\text{Gain} = \frac{1}{2} [G_L^2 / H_L + \lambda + G_R^2 / H_R + \lambda - G^2 / H + \lambda] - \gamma$$

A split is kept only if it increases this gain — built-in pruning.

What Makes It "Extreme" & Key Hyperparameters

System-Level Speed

Parallel split-finding, cache-aware access, block compression, and sparsity-aware routines.

These engineering choices are why XGBoost trains far faster than a naive gradient boosting implementation at similar accuracy.

n_estimators

Number of boosting rounds (trees)

max_depth

Max depth per tree — controls complexity

learning_rate

Shrinkage η applied to each tree's output

subsample

Row/column sampling per tree, for regularization

Advantages & Limitations

ADVANTAGES

- State-of-the-art accuracy on most tabular datasets
- Regularization and pruning reduce overfitting risk
- Fast, parallelizable, and handles missing values natively

LIMITATIONS

- Many hyperparameters — tuning takes real effort
- Less interpretable than a single decision tree
- Can still overfit small or very noisy datasets

When to Reach for XGBoost



Structured / tabular data

Competition-grade performance on rows-and-columns datasets.



Kaggle-style prediction tasks

One of the most consistently winning algorithms in ML contests.



Data with missing values

Learns a default split direction for missing entries automatically.



When you can afford to tune

Rewards careful tuning of depth, learning rate, and regularization.

Quick Start — xgboost

```
import xgboost as xgb

model = xgb.XGBRegressor(
    n_estimators=300,
    max_depth=5,
    learning_rate=0.05,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,
    random_state=42,
)

model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

Notes

- Use `early_stopping_rounds` with a validation set to pick `n_estimators`
- Lower `learning_rate` + more trees usually generalizes better
- `subsample` and `colsample_bytree` < 1 help fight overfitting

Key Takeaways

- 1 Boosts trees additively, each one fitting the previous residuals
- 2 Uses gradient AND hessian for a sharper optimization step
- 3 Regularizes tree complexity directly in the objective function
- 4 Fast, accurate, and tuning-hungry — the go-to for tabular data

Thank you